Welcome Sam Altman to Bangalore

You ask Sam Altman for 2.2 billion USD in funding, and he makes fun of you in public

He asks you where you live, and finally, after talking to you, gives you a challenging task

Collect raw Instagram data of all OpenAI followers

Answer these questions:

- Who has the maximum posts
- who has the maximum followers
- Who follows the maximum people
- How many categories (Digital creators, Non-profit foundation, etc do we have? How many people do we have?

You have 24 hours!

You then ask for data - **he laughs**!

Data Collection in Indiranagar

You hire **Vijyalaxmi Iyer** and **Sam Chandra** to help with the project. Vijyalaxmi lives all the way in **Hebbal**, and Sam stays in **HSR Layout**, so meeting at either place would be a long trip for someone. To keep things fair, you decide to set up a meeting at **Rameshwaram Café in Indiranagar**, a perfect midway point.

Three of you sit down and lay out the entire project plan. You break the work into two major parts:

- **Laxmi and Sam** will be in charge of **collecting the Instagram follower data.**
- **You** will build the Python Script that will process everything.

By the end of the meeting, everyone is aligned: Laxmi and Sam will focus on accurate data collection, and you'll focus on making sure the code can handle real-world data seamlessly. The team is ready, the plan is set, and the project officially kicks off.

Parsing Data in Pure Python

While Laxmi and Sam are collecting the real data, we continue building the core of our project using a simple demo dataset. This lets us solidify the parsing logic so that the moment the actual field data arrives, the program will already be ready to handle it.

We begin by loading the demo text file. The file contains multiple profiles, each separated by an empty line.

```
with open("finaldata.txt", encoding='utf-8') as f:
data = f.read()
```

Printing the variable shows the raw text exactly as it appears in the file.

```
data
```

The next step is to break the data into individual profile blocks. Each profile in the file is separated by two newline characters, so splitting on that pattern gives us separate units to work with.

- \n = new line
- \n\n means **blank line (double newline)** (separator between profiles)
- Used to separate **different Instagram profiles**

📌 So each Instagram profile becomes one element in the list.

```
chunks = data.split("\n\n")
chunks = [c for c in chunks if len(c)>3]
chunks
```

Output: chunks = [
"intaglobal\n1,946 posts\n6,851 followers\n262 following\nINTA\Nonprofit organization\The association of ™ professionals.\inta.campsite.bio.",
"_anujsinghal\n1,785 posts\n681K followers...",
"code_flare\n165 posts\n216 followers..."]

At this point, the list contains one chunk per profile. To examine a single profile and better understand the format, we can print an element such as:

Now, chunks = [c for c in chunks if len(c) > 3]

👉 This filters the list

Meaning:

- Loop through each c in chunks
- Keep only those where len(c) > 3

🤔 **Why len(c) > 3?**

Because after splitting:

- Some elements might be:
  - empty string ""
  - very small junk like "\n"

👉 These are useless → we remove them

🔍 **Example**
Before filtering: ['profile1 data', '', 'profile2 data', '\n', 'profile3 data']
After filtering: ['profile1 data', 'profile2 data', 'profile3 data']

```
chunks[0]
```

Output: 'intaglobal\n1,946 posts\n6,851 followers\n262 following\nINTA\nNonprofit organization\nThe association of ™ professionals.\nninta.campsite.bio ……'

This helps us see the structure of each block before writing the parsing logic.

The goal is to convert each chunk from raw text into a clean Python dictionary. For that, we define a function that trims extra whitespace, separates the lines, removes labels, and returns the meaningful parts.

```
def parse_chunk(chunk):
    chunk = chunk.strip()                                  → removes extra spaces/newlines
    lines = chunk.split("\n")                              → converts text into list of lines
    name = lines[0].replace("Name: ", "")
    posts = lines[1].replace("Posts: ", "")
    followers = lines[2].replace("Followers: ", "")
    bio = lines[3].replace("Bio: ", "")

    return {"name": name, "posts": posts, "followers": followers, "bio": bio}
```

OR

```
def parse_chunk(chunk):
        chunk = chunk.strip()                              Removes: extra \n at start/end
        sep_chunk = chunk.split('\n')                      Now the profile becomes a list
        username = sep_chunk[0]                            Directly accessing data using index
        no_of_posts = sep_chunk[1]
        no_of_followers = sep_chunk[2]
        no_of_following = sep_chunk[3]
        name = sep_chunk[4]
```

```
if(len(sep_chunk) > 5):
        type_of_page = sep_chunk[5]
        bio = "\n".join(sep_chunk[6:])        ──► means 6th index ke baad ke saare lines ko
else:                                            ek bio string bana do, line by line format mai
        type_of_page = "Unknown"
        bio = ""

# print(username, no_of_posts, no_of_followers, no_of_following, name, type_of_page, bio,
  sep="\n")        Here, sep = "\n" is used for changing seperator

return {
        "username": username,
        "no_of_posts": no_of_posts,
        "no_of_followers": no_of_followers,
        "no_of_following": no_of_following,
        "name": name,
        "type_of_page": type_of_page,
        "bio": bio
    }
```

Explanation:-

`if(len(sep_chunk) > 5):`

👉 Why?

- Some profiles have **bio + category**
- Some don't

✅ **If extra data exists:**

```
type_of_page = sep_chunk[5]
bio = "\n".join(sep_chunk[6:])
```

- 👉 **sep_chunk[5]** → category (like *Blogger, Media, Investor*)
- 👉 **sep_chunk[6:]** → remaining lines = bio

  Output: [
  'The association of ™ professionals.',
  'inta.campsite.bio'
  ]
- 👉 **"\\n".join(...)** → combines list back into text
  - "\n".join(['line1', 'line2'])
    - List ko ek string me convert karta hai
    - Har element ke beech **new line (\n)** add karta hai

  Output: The association of ™ professionals.
  inta.campsite.bio

❌ **If no extra data:**

```
type_of_page = "Unknown"
bio = ""
```

👉 Prevents errors like:
IndexError: list index out of range

Running the function on a sample chunk gives a structured dictionary containing the name, posts, followers and bio fields extracted from the original text.

So far, we have a stable method to read demo data from a file, split it into separate profile blocks, and parse each block into a structured format using only basic Python. This sets up the foundation for the next step.

```
all_chunks = []
for chunk in chunks:
    parsed_chunk = parse_chunk(chunk)
    all_chunks.append(parsed_chunk)

# print(all_chunks)
import json
s = json.dumps(all_chunks, indent=4)
print(s)
```

Output:


```
[
  {
    "username": "intaglobal",
    "no_of_posts": "1,946 posts",
    "no_of_followers": "6,851 followers",
    "no_of_following": "262 following",
    "name": "INTA",
    "type_of_page": "Nonprofit organization",
    "bio": "The association of \u2122 professionals.\ninta.campsite.bio"
  },
  {
    "username": "_anujsinghal",
    "no_of_posts": "1,785 posts",
    "no_of_followers": "681K followers",
    "no_of_following": "248 following",
    "name": "Anuj Singhal",
    "type_of_page": "Digital creator",
    "bio": "Managing Editor, CNBC-Awaaz. Most trusted and followed biz anchor in
            India\nLet\u2019s talk about finance and life\nContact:
            Singhalanuj1010@gmail.com\nhindi.cnbctv18.com/market-gurukul"
  },
 {....}
]
```

```
def parse_chunk(chunk):
    chunk = chunk.strip()
    sep_chunk = chunk.split('\n')
    username = sep_chunk[0]

    no_of_posts = int(sep_chunk[1].split(' post')[0].replace(',', ''))

    no_of_followers = float(sep_chunk[2].split(' follower')[0].replace(',', '').replace('K', '').replace('M', ''))
    if('K' in sep_chunk[2]):
        no_of_followers = int(no_of_followers * 1000)
    elif('M' in sep_chunk[2]):
        no_of_followers = int(no_of_followers * 1000000)
    else:
        no_of_followers = int(no_of_followers)


    no_of_following = float(sep_chunk[3].split(' following')[0].replace(',', '').replace('K', '').replace('M', ''))
    if('K' in sep_chunk[3]):
        no_of_following = int(no_of_following * 1000)
    elif('M' in sep_chunk[3]):
        no_of_following = int(no_of_following * 1000000)
    else:
        no_of_following = int(no_of_following)
```

```
    name = sep_chunk[4]
    if(len(sep_chunk) > 5):
        type_of_page = sep_chunk[5]
        bio = "\n".join(sep_chunk[6:])
    else:
        type_of_page = "Unknown"
        bio = ""

    # print(username, no_of_posts, no_of_followers, no_of_following, name, type_of_page, bio, sep="\n")

    return {
        "username": username,
        "no_of_posts": no_of_posts,
        "no_of_followers": no_of_followers,
        "no_of_following": no_of_following,
        "name": name,
        "type_of_page": type_of_page,
        "bio": bio
    }
```

```
all_chunks = []
for chunk in chunks:
    parsed_chunk = parse_chunk(chunk)
    all_chunks.append(parsed_chunk)

# print(all_chunks)
import json

s = json.dumps(all_chunks, indent=4)
print(s)
```

Output: [{
"username": "intaglobal",
"no_of_posts": **1946**,
"no_of_followers": **6851**,
"no_of_following": **262**,
"name": "INTA",
"type_of_page": "Nonprofit organization",
"bio": "The association of \u2122 professionals.\ninta.campsite.bio"
},
{.....}
]

Q1: Who has maximum posts

let write code to find max posts

```
max = 0
for chunk in all_chunks:
    if(max < chunk['no_of_posts']):
        max = chunk['no_of_posts']
        chunk_with_max_post = chunk
print(chunk_with_max_post)
```

Output: {'username': 'startuphub_blr', 'no_of_posts': **2300**, 'no_of_followers': 45000, 'no_of_following': 120, 'name': 'Startup Hub Bangalore', 'type_of_page': 'Media', 'bio': '🦄 News from the Silicon Valley of India\n📢 Funding alerts, Hiring trends, and Drama\n📩 DM for features\nstartuphub.blr/newsletter'}

Q2: Who has maximum follower

let write code to find max follower

```
max_follower = 0
for chunk in all_chunks:
   if(max_follower < chunk['no_of_followers']):
      max_follower = chunk['no_of_followers']
      max_no_of_follower = chunk

print(max_no_of_follower)
```

Output {'username': '_anujsinghal', 'no_of_posts': 1785, 'no_of_followers': **681000**, 'no_of_following': 248, 'name': 'Anuj Singhal', 'type_of_page': 'Digital creator', 'bio': 'Managing Editor, CNBC-Awaaz. Most trusted and followed biz anchor in India\nLet's talk about finance and life\nContact: Singhalauj1010@gmail.com\nhindi.cnbctv18.com/market-gurukul'}

Q3: Who has maximum following

let write code to find max following

```
max_following = 0
for chunk in all_chunks:
   if(max_following < chunk['no_of_following']):
      max_following = chunk['no_of_following']
      max_no_of_following = chunk

print(max_no_of_following)
```

Output: {'username': 'katiasales2220', 'no_of_posts': 107, 'no_of_followers': 219, 'no_of_following': **1401**, 'name': 'Katia Sales', 'type_of_page': 'Alfabetização e Reforço Escolar', 'bio': 'Ensino Fundamental I e II\nPreparação para Escola Militar\nPortuguês e Matemática\nCursando Psicopedagogia Estácio'}

Q4: How many categories

let write code to find categories

```
for chunk in all_chunks:
   if chunk['type_of_page'] == "Unknown":
      chunk['type_of_page'] = "Hello world"
   print(chunk['type_of_page'])
```

Output: Nonprofit organization
Digital creator
Blogger
Entrepreneur
Public Figure
Media
Food & Drink
Investor
Community
Personal Blog
Gamer
Hello world
Alfabetização e Reforço Escolar

OR

```
categories = set()
for chunk in all_chunks:
   if chunk['type_of_page'] == "Unknown":
      chunk['type_of_page'] = "Hello world"
   categories.add(chunk['type_of_page'])
print(categories, len(categories))
```

Output: {'Personal Blog', 'Investor', 'Hello world', 'Alfabetização e Reforço Escolar', 'Gamer', 'Blogger', 'Food & Drink', 'Media', 'Community', 'Digital creator', 'Nonprofit organization', 'Entrepreneur', 'Public Figure'} 13